

RoBERTa LoRA vs RoBERTa Full Fine-Tuning on Natural Language Understanding Tasks: DART, TREC

Athena Leong
CECS
Orlando, FL
at970424@ucf.edu

Nicholas Mistry
CECS
Orlando, FL
Nicholas.Mistry@ucf.edu

Keller Bonnett
CECS
Orlando, FL
ke858915@ucf.edu

Dipayan Sarkar
CECS
Orlando, FL
dipayan.sarkar@ucf.edu

Abstract—This project compares full fine-tuning of RoBERTa with parameter-efficient fine-tuning using Low-Rank Adaptation (LoRA) across two tasks: question classification with the TREC dataset and masked language modeling with the DART dataset. The experimental design boasts a new implementation but the maintains the theoretical backbone, optimizer, and training epochs, isolating the impact of trainable parameter scope. On TREC, LoRA RoBERTa achieved 97% accuracy, slightly surpassing full fine-tuning at 96%, while reducing training time by over 35%, GPU memory usage by 33%, and trainable parameters to less than 1% of the full model. These results reiterate that LoRA can deliver state-of-the-art performance with significant efficiency gains. Supplemental experiments on DART revealed lower masked token accuracy and overlap metrics, highlighting the challenges of adapting RoBERTa directly to structured data-to-text tasks and the importance of preprocessing and evaluation design. Overall, this study contributes a reproducible comparison of LoRA and full fine-tuning, quantifies efficiency tradeoffs, and identifies both its strengths and limitations.

Index Terms—TREC, LoRA, RoBERTa, BERT, DART, NLU, NLG, LLMs, BLEU, ROUGE-L

I. INTRODUCTION

The increasing scale and complexity of large language models (LLMs) have led to significant advancements in natural language understanding (NLU) tasks. However, these gains come at the cost of high computation and storage demands, even with other traditional training methods like full fine-tuning. Full fine-tuning approaches require updating and storing whole copies of model parameters for each downstream task, which fails to scale well with larger models like GPT-3 [5]. Existing alternatives like PreEmbed, PreLayer, and adapter-based tuning try to reduce this burden but often introduce inference latency and struggle to scale efficiently.

To address these limitations, Hu et al. (2021) devised Low-Rank Adaptation of Large Language Models (LoRA), a parameter-efficient training method that effectively reduces the number of trainable parameters by injecting low-rank decomposition matrices into pre-trained Transformer layers while keeping original weights frozen [5]. This allows for faster training and lower memory consumption without needing to duplicate the entire model. LoRA has demonstrated up to a 25% training speedup and a substantial reduction in memory

usage, making it a compelling alternative to the other training methods, in particularly full fine-tuning [5].

This study investigates the effectiveness of LoRA in adapting a transformer-based model on structured NLU tasks. In particular, the RoBERTa-base model will be trained with a version of LoRA made from scratch, created by following Hu et al.’s logic and supplemented by their GitHub [1]. The model will be trained using this new implementation of LoRA on the TREC dataset, and those findings will be supplemented with training on a modified version of the DART dataset. To suit the nature of sequence classification and NLU tasks, the DART dataset has been reformulated into masked language modeling (MLM) tasks to better align with the RoBERTa or “Robustly Optimized BERT Approach” architecture [2]. The performance and resource efficiency of LoRA-based training will be compared against a full fine-tuning baseline on the same MLM-transformed DART and basic TREC datasets. The aim is to evaluate whether LoRA can achieve comparable performance while offering improvements in training efficiency and parameter economy.

The contributions of this work are as follows:

- Construction of an MLM-compatible version of the DART dataset tailored for RoBERTa’s NLU capabilities
- A fresh version of the LoRA training model created from scratch with PyTorch and HuggingFace integrations
- Implementation and evaluation of LoRA-based training on RoBERTa-base, using full fine-tuning as a benchmark
- Analysis of performance metrics and resource utilization to assess the practical benefits of LoRA in two different structured NLU adaptations (TREC and DART)

The remainder of this report is organized as follows:

- **Related Work** reviews related work in parameter-efficient fine-tuning, RoBERTa model architecture, and TREC/DART dataset tasks
- **Methodology** details the dataset pre-processing and experimental setup
- **Results** presents the results and comparative analysis
- **Discussion** discusses key findings and limitations
- **Conclusions** concludes with potential directions for fu-

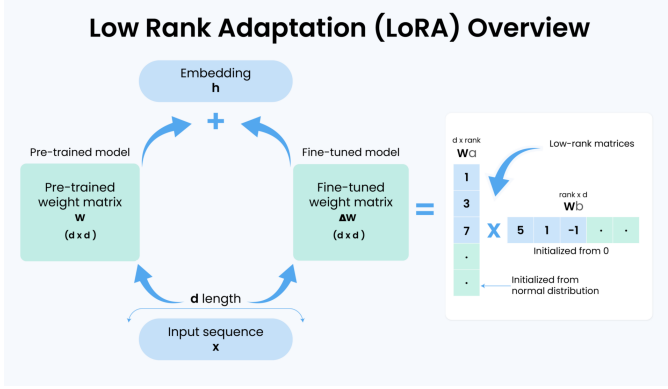


Fig. 1. LoRA Overview Diagram [9]

ture research in LoRA-based adaptation strategies and a summary of the paper’s contributions

II. RELATED WORK

This study builds on the work of Hu et al. (2021), who introduced LoRA 1 as a parameter-efficient alternative to full fine-tuning [5]. They observed that weight updates during adaptation tend to have low intrinsic rank, and proposed injecting trainable low-rank matrices into frozen pre-trained weights. This significantly reduces the number of trainable parameters while maintaining performance, enabling faster training and lower memory usage. Unlike adapter-based methods such as PreEmbed and PreLayer, LoRA introduces no additional inference latency due to its linear design, and scales effectively to large models like GPT-3. Its linear design allows the injected matrices to be merged with the frozen weights at deployment, keeping inference overhead negligible. Using adaptive optimizers like AdamW further lowers the hardware barrier to entry, as only the small injected matrices require optimization; gradient computation and optimizer state tracking for the full model are unnecessary. The original paper demonstrated LoRA’s effectiveness on tasks like natural language inference and question answering, achieving up to 10,000× parameter reduction with comparable accuracy [5]. Its modularity also facilitates efficient task-switching, reducing storage and overhead when deploying models across multiple domains.

The following equation [5] for full fine-tuning represents the standard maximum training objective for LLMs, seeking to optimize the model parameters Φ , by maximizing the log-likelihood of generating the correct output sequences y given input x . For each (x, y) in dataset Z , the model sums the log-probabilities of each token y_t , conditioned on x and all previous tokens $y_{<t}$. This encourages the model to assign high probability to the correct next token at each step, effectively learning to generate sequences that resemble the training data.

$$\max_{\Phi} \sum_{(x,y) \in Z} |y| \sum_{t=1}^{|y|} \log (P_{\Phi}(y_t | x, y_{<t}))$$

To support parameter-efficient fine-tuning, this objective can be modified with LoRA [5]. Instead of updating the full model Φ , it assumes a frozen base model Φ_0 and introduces a learnable adjustment function $\Delta_{\Phi}(\Theta)$, parameterized by Θ . The model now optimizes Θ such that the combined output of the base model and the delta function improves the model’s predictions. The log-likelihood is computed over the sum of the base model’s output p_{Φ_0} and the adjustment $\Delta_{\Phi}(\Theta)$. This enables efficient adaptation to new tasks or domains without the computational cost of full model retraining.

$$\max_{\Theta} \sum_{(x,y) \in Z} |y| \sum_{t=1}^{|y|} \log (p_{\Phi_0} + \Delta_{\Phi}(\Theta)(y_t | x, y_{<t}))$$

To validate our implementation of LoRA to the established LoRA implementation, we seek to confirm an exercise. By computing subspace similarity of different trained LoRA models varied by rank and seeds, we try to prove low ranks do not cover a more meaningful subspace. Using the equation defined in the original LoRA paper by Hu et al. (2021), computation of the subspace similarity of our implementation can be used as a sanity check if they are consistent with the original paper’s findings. The equation is derived from the singular value decomposition of both weight update matrices (rank 8 and rank 64) then computing the Frobenius squared norm which measures subspace overlap; this is normalized to a value between 0 and 1.

$$\phi(A_{r=8}, A_{r=64}, i, j) = \frac{\|U_{A_{r=8}}^{i\top} U_{A_{r=64}}^j\|_F^2}{\min(i, j)} \in [0, 1] \quad (1)$$

This equation is used on the same pretrained model trained using LoRA on the same dataset. The motivation being, that when rank increases, the number of trainable parameters increases too. What the authors suggest is that low ranks have a similar update matrix to an update matrix with higher ranked. Since they are similar, a model can be trained with fewer parameters and return similar performance to a model trained with more parameters.

The figures 2 3 demonstrate that the top singular vectors overlap which is consistent with the LoRA findings Hu et al. (2021). The paper also suggests that this overlapping vector is significant for the new task.

To show that increasing rank does not equal a substantial change in metrics, a simple demonstration can be performed by varying rank and measuring accuracy. From the TREC dataset, increasing rank does not change the metrics significantly, indicating that even at a low rank, the subset of parameters are able to produce an update matrix that is similar in performance to an update matrix with even more parameters. In this demonstration, the lowest rank model trained had the highest performance, but it is stated that there is no association between ideal rank and model size. The authors suggest that the association could be more related to the task at hand.

Several related approaches have explored efficient adaptation of pre-trained models. Adapter layers (Houlsby et al.,

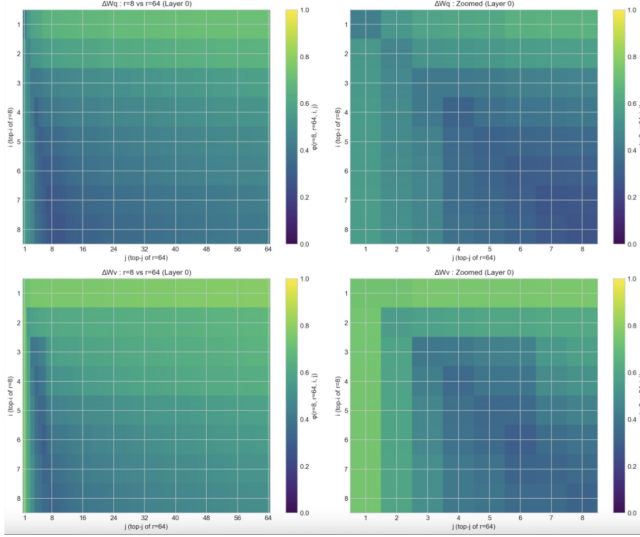


Fig. 2. Subspace Similarity of Rank 8 and Rank 64

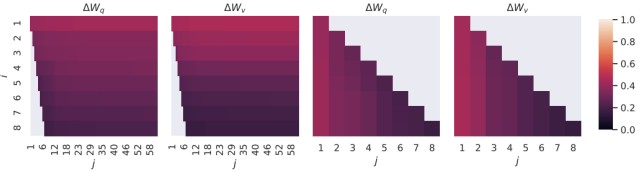


Fig. 3. LoRA Subspace Similarity rank 8 and rank 64, Hu et al. (2021)

2019) insert small bottleneck modules between layers, while prefix-tuning (Li & Liang, 2021) prepends trainable vectors to attention inputs [5]. Although these methods reduce training costs by pruning layers or exploiting multi-task settings, they often introduce runtime overhead or require architectural modifications. Adapter layers also come with extra compute that cannot be skipped and is unparallelizable, so each layer has to be computed sequentially. Prefix tuning is just one attempt to optimize forms of input layer activations since direct prompt optimization is out of scope. However, it is still difficult to optimize and at times, makes the prompt perform worse since it reduces the sequence length available to process tasks. LoRA’s advantage lies in its simplicity, modularity, and compatibility with existing architectures without altering inference behavior.

To evaluate LoRA’s performance in a structured Natural Language Understanding setting, RoBERTa-base is trained and tested on TREC and a modified version of the DART dataset, a benchmark for data-to-text generation [3]. By re-implementing

TABLE I
TREC DATASET RESULTS (SEED = 42)

Rank	Accuracy	Loss
1	0.95	0.2317
8	0.94	0.2561
64	0.946	0.2396

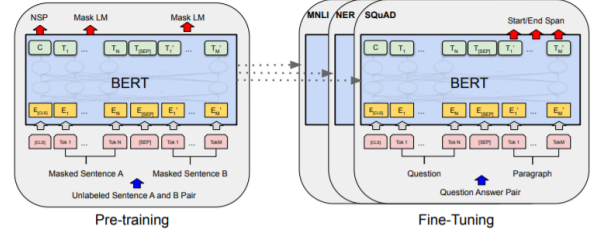


Fig. 4. BERT and RoBERTa Architecture [8]

LoRA and adapting the DART dataset for Masked Language Modeling (MLM), this work aims to contribute a reproducible framework for evaluating parameter-efficient training methods on structured NLU tasks. It complements existing literature by extending LoRA beyond its original scope, training and testing performance on new tasks, and providing empirical comparisons with full fine-tuning under controlled conditions. The findings aim to inform future research on scalable, efficient adaptation strategies for transformer-based models.

METHODOLOGY

A. Model Architecture

This project extends LoRA’s application by implementing it from scratch and applying it to “Robustly Optimized BERT Approach” (RoBERTa), a transformer-based encoder model introduced by Meta AI researchers Liu et al. (2019) [2]. RoBERTa builds on the original “Bidirectional Encoder Representations from Transformers” (BERT) model by optimizing training dynamics, removing the next sentence prediction objective, increasing batch size and training duration, and using dynamic masking. It retains the core transformer architecture with 12 layers, 768 hidden units, and 12 attention heads, making it well-suited for masked language modeling tasks [2] [5]. RoBERTa-base is a 125M parameter model and is still a competitively popular pre-trained model today. Since BERT is an encoder model, it is unable to be trained and test on the DART dataset as is. It handles NLU (TREC) and MLM tasks efficiently, but experiences difficulty when attempting Natural Language Generation (NLG) tasks; transformer-based decoders like the GPT family perform better with NLG tasks.

To enable parameter-efficient fine-tuning, the Low-Rank Adaptation method is integrated into RoBERTa’s attention layers. LoRA introduces trainable low-rank matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$ into the attention projection layers, where $r \ll \min(d, k)$. These matrices are inserted in parallel with the frozen pre-trained weights, allowing the model to learn task-specific adaptations without modifying the original parameters. During inference, the low-rank updates are merged with the frozen weights, ensuring no additional latency. The process is then optimized using adaptive optimizers like AdamW [5], resulting in faster and less intensive training.

AdamW [11] is the optimization algorithm used to help train RoBERTa-base. A variant of Adam (Adaptive Move-

ment Estimation), it keeps track of past gradients for smooth updates, adaptively scales updates for each parameter, and regularizes weight decay to prevent overfitting. It differs from Adam by decoupling weight decay from the gradient update, improving training and generalization. This study uses AdamW for training RoBERTa because it helps large models converge faster and generalize better. In the implementation, AdamW is the engine that updates model parameters during training, applying weight decay to keep weights from growing too large, and ensuring LoRA layers and classifier weights (those left trainable) are updated efficiently.

B. Implementation Details

The model was implemented using the PyTorch deep learning framework, with the HuggingFace Transformers library providing pre-trained weights and tokenizer methods for RoBERTa-base. The LoRA program and modules were implemented from scratch and injected into the query and value projection layers of the multi-head self-attention. LoRALayer injects a low-rank adapter into a frozen linear layer, scaling the output by α/r to control update magnitude such that the original weights are frozen and only *lora_A* and *lora_B* can be trained. The self-attention forward function of RoBERTa is overridden, wrapping RobertaForMaskedLM and injecting LoRA into attention layers (query and value) adding the LoRA output to the frozen base output at runtime.

This implementation is more simplified in comparison to the LoRA paper, but is still efficient enough to handle the assigned tasks. Here, the only injection points are the query and value projection layers, but Hu et al. [5] inject the query, key, value, and output layers. Additionally, runtime addition occurs instead of at inference time. Their implementation of LoRA is also wrapped in PEFT libraries, PyTorch, and HuggingFace [1], while this study purely uses PyTorch and HuggingFace. These differences generally improve usability and functionality due to the modularity of using libraries, so this implementation from scratch is still viable.

Additionally, the from-scratch implementation selectively froze parameters in the RoBERTa-base model so that only the LoRA adapter layers and the classifier head remained trainable. This was achieved by iterating through all named parameters and setting `requires_grad = False` for any parameter whose name did not include "*lora_*" or "*classifier*". The effect is that the large pretrained backbone of RoBERTa is kept fixed, while the lightweight low-rank adapters and the final classification layer are updated during training. Compared to full fine-tuning, where every parameter in the model is adjusted, this freezing strategy reduces the number of trainable parameters, lowers memory and compute requirements, and makes training more efficient. The key difference lies in scope: full fine-tuning adapts the entire model to the new task, while freezing parameters focuses learning capacity on a small, targeted subset of layers, which is the essence of parameter-efficient fine-tuning methods like LoRA.

The following hyper-parameters were used for training:

- Learning rate: 2e-4 (LoRA), 5e-5 (full fine-tuning)

- Tokenizer length: 128
- Batch size: 16
- Epochs: 50 for TREC, 100 for DART
- LoRA rank (*r*): 8
- Dropout: 0.1
- Weight decay: 0.01
- Optimizer: AdamW

Training was conducted using mixed precision (fp16) to reduce memory usage and accelerate computation. Gradient clipping was applied with a max norm of 1.0 to stabilize training. For LoRA, only low-rank matrices are updated, while all other parameters remained frozen. In contrast, the full fine-tuning baseline updated all model parameters.

C. Dataset

To evaluate LoRA's performance in a structured NLU setting, RoBERTa-base is trained and tested on both the TREC datasets and a modified version of the DART generation dataset [3], a benchmark for decoder models to test their language generation capabilities. TREC (Text REtrieval Conference) [10] is a benchmark dataset for question classification, containing 5,500 training questions and 500 test question, divided into 6 coarse labeled categories (Description, Entity, Abbreviation, Human, Location, Numeric) and 50 fine-grain labeled subcategories. The benchmark is designed for evaluating question classification tasks in natural language processing; for the purpose of analyzing training, only coarse labels (TREC-6) were used for the paper. It is a small but highly influential dataset where the goal is to predict the correct category for each question. This helps evaluate and train models to understand the type of information being requested i.e. in chatbots and search engines [10].

Listing 1. Example input from TREC 'train'

```

1 {
2   'text': 'How did serfdom develop in and then
          leave Russia ?',
3   'coarse_label': 2,
4   'fine_label': 26
5 }
```

DART (Data Record to Text) is significantly larger and more complex compared to other generation datasets like E2E [5]. Originally designed for sequence-to-sequence tasks, DART contains structured triplesets and corresponding natural language descriptions. The corpus includes high-quality sentence annotations and inputs are sets of entity-relation triples following a tree-structured ontology. It consists of 82191 examples across different domains with each input being a semantic tripleset derived from data records in tables and the tree ontology of table schema, annotated with sentence description that covers all facts in the triple set [7].

Listing 2. Example Data Record from DART

```

1 {
2   "tripleaset": [
3     ["Ben Mauk", "High school", "Kenton"],
4     ["Ben Mauk", "College", "Wake Forest
          Cincinnati"]
  ]
}
```

```

5     ],
6     "subtree_was_extended": false,
7     "annotations": [
8         {
9             "source": "WikiTableQuestions_lily",
10            "text": "Ben Mauk, who attended Kenton
                    High School, attended Wake Forest
                    Cincinnati for college."
11        }
12    ]
13 }

```

To align with RoBERTa’s architecture, the dataset was reformulated into an MLM task by masking tokens within the reference text. This means instead of sentence generation, the dataset will assess the model’s ability to predict the masked token. The reference text is created by extracting the first triple in the triplesets and replacing it with RoBERTa’s `mask` token and storing the masked sentence and original object as the prediction target. Then in the preprocess function, the masked input is tokenized and a MLM-style label is attached to the index of the `mask` token. Each example in the training set is converted into token IDs, attention masks, and label tensors to prepare the dataset for training. To evaluate the trained model using the masked test set, the predicted token is compared to the ground truth. This transformation reframes the task as a token prediction problem, allowing the bidirectional encoder to process the tasks as NLU. The resulting MLM DART dataset retains the semantic richness of the original while making it suitable for evaluating adaptation in a non-generative, encoder-only setting.

D. Experimental Setup

Training and evaluation were conducted on Google Colab Pro and Cuda machines using a Tesla T4 GPU with 16GB VRAM to maintain consistency throughout the study. The software environment and applications used included Python 3.10, PyTorch 2.1.0, Transformers 4.35.0, CUDA 11.8 (for local machine reproducibility), and datasets and tokenizers from HuggingFace. This also includes RoBERTa-base access and integration.

The same codebase can be tested on local machines equipped with CUDA-enabled environments to confirm reproducibility across hardware configurations.

Training and testing setup include:

- Optimizer: AdamW with learning rates $2e-5$ (LoRA), $5e-5$ (full fine-tuning)
- Tokenizer length: 128
- Batch size: 16 for training, 32 for testing
- Epochs: 50 for TREC, 100 for DART
- Loss: Cross-entropy
- Loop:
 - Training loop accumulates loss per epoch
 - Evaluation loop computes predictions, accuracy, and average loss

E. Evaluation Metrics

Model performance was evaluated using the following metrics:

- Classification Accuracy (TREC NLU): The percentage of correctly predicted question labels, serving as the primary indicator of classification performance on the TREC dataset
- Masked Token Accuracy (DART MLM): The percentage of correctly predicted masked tokens, serving as the primary indicator of NLU performance on the MLM DART dataset
- Cross-Entropy Loss (Both): The average token-level loss across the validation set, used to monitor convergence and generalization
 - TREC: loss is computed at the sequence classification level
 - DART: loss is computed at the token level for masked predictions
- Parameter Efficiency (Both): The number of trainable parameters during adaptation, highlighting memory and storage benefits of LoRA
- Training Time (TREC): Measured per epoch to assess throughput
- GPU Memory Usage (Both): Peak VRAM consumption during training

These metrics collectively capture both the effectiveness and efficiency of LoRA compared to full fine-tuning. By isolating the adaptation mechanism while keeping the model and dataset constant, the evaluation provides a controlled comparison of resource-performance trade-offs.

RESULTS

F. TREC Results

The results for the TREC training and testing are in Table 1. The results show that LoRA RoBERTa slightly outperforms full fine-tuning on TREC in 50 epochs, while being far more efficient in parameters, memory usage, and training time. Compared to baselines from the original TREC papers (95-95% accuracy with traditional classifiers and 97% with modern transformers), the from-scratch LoRA implementation achieves state-of-the-art performance with a fraction of the cost.

For Fine-Tuned RoBERTa, the model exhibits a strong performance both in training and testing, consistent with expectations from a fully fine-tuned transformer on TREC. The cost, however, is exceptionally high: all 124.6M parameters are updated, requiring more compute and memory. It achieves test accuracy of 96% and average Cross-Entropy Loss of 0.27, but GPU usage is high at 3.6 GB.

For LoRA RoBERTa, LoRA not only reduces resource usage dramatically but also improves accuracy and convergence. This suggests that LoRA adapters are sufficient to capture task-specific features without disturbing the pretrained backbone. There is a 1% increase in accuracy to 97% and decrease in loss of about 50% to 0.1353. It outperforms full fine-tuning

without needing to train as many parameters; it only trains 0.89M parameters (.71% of the total) and uses 2.4GB of GPU (a decrease of about 33%). It also converges 65% faster and generalizes better as training time has been reduced and there is lower loss.

Model	Accuracy	Loss	Training Time	Trainable Params	GPU
Fine-Tuned RoBERTa	96%	0.27	5812 sec (97 min)	124.6 M (100%)	3.6 GB
LoRA RoBERTa	97%	0.1353	3789 sec (64 min)	0.89M (.7%)	2.4 GB

TABLE II
TREC DATASET RESULTS

G. DART Results

Model	Accuracy	Trainable Params	GPU
Fine-Tuned RoBERTa	7.12%	124.6 M (100%)	2.50 GB
LoRA RoBERTa	6.26%	284,912 (.22%)	1.53 GB

TABLE III
DART MLM DATASET RESULTS

The evaluation of RoBERTa (Table 2) with LoRA adapters on the DART masked language modeling dataset highlights both the efficiency gains of parameter-efficient fine-tuning and the challenges of adapting to this task. In terms of resource usage, the model required only 294,912 trainable parameters, representing just .22% of the total 126 million parameters. This reduction translated into a compact checkpoint size of roughly 500 MB, peak GPU memory usage of only 1.53 GB. These results confirm that LoRA delivers substantial savings in computational cost and storage even on a different task dataset.

For RoBERTa-base fine-tuned on the DART MLM dataset, its masked token accuracy was very low compared to expectations (7.12%) which is still higher than LoRA’s performance. This suggests that the fine-tuning setup struggled to adapt to a MLM DART input, and LoRA struggled more than full fine-tuning. The GPU usage was almost twice that of LoRA, confirming the setup was computationally less efficient than LoRA’s. 100% of the trainable parameters were used. Its performance, while underwhelming, means that adjustments must be made so that the model aligns better with MLM in the future.

Despite these efficiency benefits, the performance metrics reveal a significant gap relative to expectations for masked language modeling. The model achieved a masked token accuracy of 6.26%, which is far below the levels typically reported for pretrained RoBERTa on standard MLM benchmarks, where accuracies often exceed 50%. Similarly, average BLEU and ROUGE-L scores were 0.063 for LoRA and 0.000 and 0.0074 (respectively) for fine-tuning, indicating minimal overlap between predicted and reference tokens. These low values suggest that the model struggled to generalize effectively to the DART dataset. One contributing factor may be the 25% of examples that were skipped during evaluation, which could reflect issues with tokenization, masking strategy, or dataset alignment with the pretrained vocabulary.

Taken together, these results demonstrate that while LoRA provides clear efficiency advantages, its performance on DART

highlights the importance of careful preprocessing and evaluation design. The low accuracy and overlap metrics point to potential mismatches between the dataset and the pre-trained model’s distribution, or to overly aggressive masking strategies that hinder learning. Future work should investigate adjustments such as refining the masking pipeline, increasing the LoRA rank, or extending training epochs to improve convergence. In this way, the experiment underscores both the promise of LoRA for efficient adaptation and the need for robust evaluation practices when applying it to challenging datasets like DART.

III. DISCUSSION

The experiments comparing full fine-tuning of RoBERTa with LoRA adapters across the TREC classification task and the supplemental DART masked language modeling task highlight both the strengths and trade-offs of parameter-efficient fine-tuning. On TREC, LoRA RoBERTa achieved slightly higher accuracy (97% vs. 96%) while reducing training time by over 35%, GPU memory usage by a third, and trainable parameters to less than 1% of the full model. This demonstrates that LoRA can match or even surpass full fine-tuning in performance while offering substantial efficiency gains. In contrast, results on DART revealed that while LoRA maintained its efficiency advantages, masked token accuracy and overlap metrics (BLEU, ROUGE-L) were far below expectations. This discrepancy underscores the importance of dataset alignment and evaluation design when applying LoRA to generative or token-level tasks.

A key limitation of this study is the difficulty of directly comparing LoRA and full fine-tuning across two very different tasks. TREC is a well-established classification benchmark, whereas DART is a structured data-to-text dataset that required adaptation into a masked language modeling format. This conversion introduced complexity and may have contributed to the low performance observed. Additionally, the evaluation metrics for DART (BLEU, ROUGE-L) are not ideal for masked token prediction, limiting the interpretability of results. Thus, while the TREC experiments provide a clean comparison, the DART results should be viewed as supplemental and exploratory rather than definitive.

Several lessons emerged from this work. First, LoRA is highly effective for classification tasks, offering efficiency without sacrificing accuracy. Second, evaluation pipelines must be carefully tailored to the nature of the dataset; metrics that work well for classification may not translate to generative tasks. Third, preprocessing and dataset conversion are critical—without proper alignment between model architecture and task format, performance can degrade sharply. Finally, the experiments reinforced the value of parameter-efficient fine-tuning as a practical alternative to full fine-tuning, especially when computational resources are limited.

The project presented multiple challenges. One major hurdle was recognizing that RoBERTa, as a masked language model, could not be applied directly to DART without converting the dataset into a masked format. This required designing

a preprocessing pipeline that introduced masks in a way consistent with RoBERTa-base’s training objective. Another challenge was the technical complexity of inserting LoRA layers correctly into the RoBERTa architecture. Ensuring that adapters were placed in the right attention projections and that only the intended parameters remained trainable demanded careful inspection of the model’s parameter names and structure. Freezing parameters was also tricky: it was essential to verify that the backbone was frozen while LoRA and classifier layers remained active. These challenges were overcome through iterative debugging, parameter inspection, and validation checks, ultimately leading to a robust experimental setup that allowed meaningful comparisons between LoRA and full fine-tuning.

IV. CONCLUSION

This work provides a systematic comparison between full fine-tuning of RoBERTa and parameter-efficient fine-tuning using LoRA adapters across two distinct tasks: question classification with the TREC dataset and masked language modeling with the DART dataset. The study demonstrates that LoRA can achieve state-of-the-art accuracy on TREC while requiring less than 1% of the trainable parameters, reducing training time by over a third, and lowering GPU memory usage by approximately 33%. These results highlight LoRA’s ability to deliver both efficiency and performance in classification tasks. In addition, the experiments extend LoRA evaluation to the DART dataset, offering supplemental insights into its behavior on generative masked language modeling tasks. While performance on DART was limited, the attempt underscores the challenges of adapting pretrained models to datasets outside their original training distribution and provides valuable lessons for future research. Overall, the contributions of this work lie in presenting a clean, reproducible experimental setup, quantifying efficiency gains, and identifying both the strengths and limitations of LoRA compared to full fine-tuning.

Future research would be refining evaluation pipelines and expanding the scope of parameter-efficient fine-tuning. For masked language modeling tasks such as DART, further work is needed to design preprocessing strategies that better align with the model’s training objective, including improved masking schemes and tokenization adjustments. Exploring higher-rank LoRA adapters, hybrid approaches that combine LoRA with partial fine-tuning, or multi-task training could also help bridge the performance gap observed in generative tasks. Beyond TREC and DART, applying LoRA to larger and more diverse datasets would provide a broader understanding of its generalization capabilities. Finally, incorporating additional evaluation metrics—such as per-class accuracy, confusion matrices, or token-level perplexity—would yield deeper insights into model behavior. By pursuing these directions, future work can strengthen the case for LoRA as a practical and scalable alternative to full fine-tuning across a wide range of NLP applications.

REFERENCES

- [1] Microsoft, “Microsoft/Lora: Code for loralib, an implementation of ‘Lora: Low-rank adaptation of large language models,’” GitHub, <https://github.com/microsoft/LoRA> (accessed Nov. 18, 2025).
- [2] Y. Liu et al., “Roberta: A robustly optimized Bert pretraining approach,” arXiv.org, <https://arxiv.org/abs/1907.11692> (accessed Nov. 6, 2025).
- [3] L. Nan et al., “DART: Open-domain structured data record to text generation,” arXiv.org, <https://arxiv.org/abs/2007.02871> (accessed Nov. 6, 2025).
- [4] Montinger, “Montinger/Transformer-Workbench: Playground for transformers,” GitHub, <https://github.com/Montinger/Transformer-Workbench> (accessed Nov. 6, 2025).
- [5] Hu, Lora: Low-rank adaptation of large LAN- guage models Edward Hu and Yelong Shen, <https://arxiv.org/pdf/2106.09685> (accessed Nov. 6, 2025).
- [6] M. Dittgen, “Implementing Lora from scratch,” Medium, <https://medium.com/data-science/implementing-lora-from-scratch-20f838b046f1> (accessed Nov. 6, 2025).
- [7] Yale-LILY, “Yale-lily/dart: Dataset for NAACL 2021 paper: ‘DART: Open-domain structured data record to text generation,’” GitHub, <https://github.com/Yale-LILY/dart> (accessed Nov. 18, 2025).
- [8] N. AI, “Introducing Roberta Base Model: A comprehensive overview,” Medium, https://medium.com/@marketing_novita.ai/introducing-roberta-base-model-a-comprehensive-overview-330338afa082 (accessed Nov. 18, 2025).
- [9] “Low-rank adaptation: A closer look at Lora,” Coralogix, <https://coralogix.com/ai-blog/low-rank-adaptation-a-closer-look-at-lora/> (accessed Nov. 18, 2025).
- [10] C. C. Group, “CogComp/TREC · Datasets at hugging face,” CogComp/trec · Datasets at Hugging Face, <https://huggingface.co/datasets/CogComp/trec> (accessed Nov. 20, 2025).
- [11] “ADAMW,” AdamW - Cornell University Computational Optimization Open Textbook - Optimization Wiki, <https://optimization.cbe.cornell.edu/index.php?title=AdamW> (accessed Nov. 20, 2025).